GPUMODE Lecture Study Notes: GPU Observability

Core Problem the Lecture Addresses

This lecture studies how to make GPU systems more observable and more extensible. The speaker frames modern GPU stacks as powerful but opaque systems: user applications, CUDA runtimes, GPU kernel drivers, firmware, and device execution all interact, but many of the most important decisions are hidden from the engineer.

The central problem is that GPU workloads have become diverse enough that fixed, black-box policies are no longer sufficient. A vector search engine, an LLM prefill kernel, an LLM decode phase, and a distributed training workload can have very different memory access patterns, scheduling needs, latency sensitivity, and CPU–GPU interaction patterns. A single hard-coded policy inside the driver, runtime, or firmware may work acceptably for one workload but poorly for another.

> The lecture's main mental model is that GPU systems need programmable observability and programmable policy hooks, analogous to how eBPF made the Linux kernel dynamically inspectable and extensible.

The lecture focuses on using eBPF-style mechanisms for GPUs. There are two major directions:

1. **Device-side eBPF**: compile eBPF-like instrumentation into PTX and inject it into GPU kernels so that one can observe fine-grained device execution.
2. **Driver-side eBPF**: add programmable hooks to GPU kernel drivers so that one can observe and customize policies such as Unified Virtual Memory migration, prefetching, eviction, and GPU channel scheduling.

The practical goal is not merely to profile a kernel after the fact. The broader goal is to create a cross-layer observability and extensibility interface that can correlate CPU events, CUDA runtime events, GPU driver events, and GPU device events.

Required Background

GPU Software Stack

A GPU application is not a single monolithic program. It passes through a layered stack:

1. **User-space application**: examples include PyTorch, vLLM, llama.cpp, vector search engines, custom CUDA applications, and distributed training frameworks.
2. **User-space GPU runtime and libraries**: examples include CUDA runtime libraries, CUDA driver APIs, cuBLAS, cuDNN, NCCL, and framework-specific dispatch layers.
3. **GPU kernel driver**: the Linux kernel driver manages GPU memory, command submission, scheduling objects, page faults, Unified Virtual Memory, and interactions with the device.
4. **GPU firmware and hardware**: firmware participates in scheduling and device-side control. The device executes thread blocks on SMs and manages memory hierarchy effects through caches, HBM, and internal schedulers.

A simplified model is:

$$\text{Application} \rightarrow \text{CUDA Runtime} \rightarrow \text{GPU Driver} \rightarrow \text{Firmware} \rightarrow \text{SM Execution}$$

This stack creates many places where visibility can be lost. A CUDA kernel may be launched by the CPU at time t_{launch}, accepted by the runtime at time t_{runtime}, submitted to a GPU queue at time t_{submit}, and only later actually have its thread blocks scheduled onto SMs at time t_{SM}.

$$t_{\text{launch}} \leq t_{\text{runtime}} \leq t_{\text{submit}} \leq t_{\text{SM}} \leq t_{\text{finish}}$$

A profiler that only reports kernel start and end time may miss where the delay actually occurred.

GPU Workload Diversity

The lecture emphasizes that GPU workloads do not behave uniformly. Different workloads can produce radically different memory access traces.

For example:

- **LLM prefill** often performs large dense attention and GEMM operations with relatively structured memory access.

- **LLM decode** may be latency-sensitive and dominated by smaller operations, KV-cache reads, synchronization, and scheduling overhead.
- **Vector search** can exhibit irregular access patterns depending on the index structure and query distribution.
- **Training** workloads may be throughput-oriented and may use large batches, activation checkpointing, gradient synchronization, and optimizer state movement.
- **Multi-tenant inference** may mix latency-critical and throughput-oriented workloads on the same GPU.

A useful abstraction is a time-address memory trace. Let $a(t)$ denote the memory address accessed at time t. Different workloads produce different shapes in the (t, a) plane:

$$\mathcal{T} = \{(t_i, a_i)\}_{i=1}^{N}$$

where t_i is an access timestamp and a_i is the accessed virtual or physical address. Sequential scans, sparse jumps, repeated KV-cache reads, and randomized vector search accesses all produce different patterns.

Visual Concept

Draw time on the horizontal axis and address space on the vertical axis. Show one workload as a narrow diagonal stripe, another as scattered random points, and another as repeated bands corresponding to reused KV-cache regions.

Unified Virtual Memory

Unified Virtual Memory, or UVM, allows CPU and GPU code to share a virtual address space. Pages may physically reside in CPU memory or GPU memory. When a GPU accesses a page that is not resident on the GPU, a GPU page fault occurs.

A simplified UVM page fault path is:

GPU load $\rightarrow$ page fault $\rightarrow$ driver fault handler $\rightarrow$ CPU-to-GPU migration $\rightarrow$ resume GPU execution

Let P be a page and let $r(P, t)$ be its residency at time t:

$$r(P, t) \in \{\text{CPU}, \text{GPU}\}$$

A GPU access to page P at time t causes a fault when:

$$r(P,t) = \text{CPU}$$

The total penalty of a UVM miss can be modeled as:

$$T_{\text{fault}} = T_{\text{trap}} + T_{\text{driver}} + T_{\text{migration}} + T_{\text{resume}}$$

where $T_{\text{migration}}$ may dominate if large pages or many pages must be moved across PCIe or NVLink.

Existing GPU Profiling and Instrumentation Tools

The lecture compares several tool classes.

CUPTI

CUPTI is NVIDIA's profiling and tracing interface. Tools such as NVIDIA profilers can use CUPTI to obtain runtime, driver, activity, and performance metric information.

Conceptually, CUPTI exposes an event stream:

$$E_{\text{CUPTI}} = \{(e_i, t_i, m_i)\}_{i=1}^{N}$$

where e_i is an event type, t_i is a timestamp, and m_i is metadata. CUPTI is convenient because applications usually do not need source changes. However, it exposes a fixed set of events and does not necessarily provide programmable fine-grained device-side logic.

NVBit and Dynamic Binary Instrumentation

NVBit-like tools operate by modifying GPU binary code before execution. The general idea is:

Original GPU code $\rightarrow$ instrumented GPU code $\rightarrow$ execution with inserted probes

This enables instruction-level or thread-level tracing, but it can introduce overhead and requires careful binary rewriting.

Nsight Compute and Nsight Systems

Nsight Compute is commonly used for kernel-level performance analysis. Nsight Systems provides broader timeline-level tracing. They are extremely useful, but the lecture's point is that they are generally profiler tools, not programmable operating-system-like extensibility mechanisms.

Main Concepts

The Two Problems: Invisibility and Inflexibility

The lecture identifies two major issues.

Invisibility

Many important GPU-system states are hidden:

- CUDA runtime internals may be closed source.
- GPU driver internals are not always easily inspectable.
- Firmware scheduling decisions are largely opaque.
- Device execution is difficult to correlate with host-side events.
- UVM page migration decisions are often hard to explain from the application level.
- Standard profilers expose predefined events rather than arbitrary programmable observations.

The missing information includes questions such as:

- Which thread blocks actually reached which SMs?
- Was a kernel delayed because launch overhead was high, or because blocks waited for SM resources?
- Which UVM pages were evicted and why?
- Which memory regions are repeatedly faulting?
- Which CPU call chain triggered a GPU kernel that later produced device-side stalls?

Inflexibility

Many GPU policies are fixed:

- UVM eviction and prefetch policies are largely hard-coded.
- Scheduling policies may use fixed priorities, time slices, or round-robin behavior.
- GPU kernel scheduling behavior is difficult to customize.
- User-space frameworks can work around some policies, but they usually cannot safely modify low-level driver behavior.

The lecture argues that driver-level hooks are valuable because the driver has global visibility across processes, tenants, memory objects, and GPU scheduling state.

What eBPF Provides on CPUs

eBPF provides safe, verified, dynamically loaded programs inside the Linux kernel. An eBPF program is commonly written in restricted C, compiled into eBPF bytecode, verified by the kernel, and attached to a hook point.

A conceptual pipeline is:

$$\text{C source} \rightarrow \text{eBPF bytecode} \rightarrow \text{verifier} \rightarrow \text{kernel hook} \rightarrow \text{runtime execution}$$

The key pieces are:

- **Programs**: small restricted programs that execute at hook points.
- **Verifier**: checks safety before execution.
- **Maps**: key-value storage shared between kernel and user space.
- **Helpers**: controlled API calls that expose safe kernel functionality.
- **Hooks**: tracepoints, kprobes, uprobes, networking hooks, security hooks, and scheduler hooks.

The safety requirement is essential. eBPF is not arbitrary kernel module loading. It prevents common dangerous behavior such as illegal memory access and unbounded loops.

Why eBPF Is Attractive for GPUs

The GPU world has a similar need:

- dynamic instrumentation without recompiling the original application;
- low-overhead tracing;
- programmable logic at observation points;
- safe extension of privileged driver behavior;
- maps for sharing observations across CPU and GPU;
- hooks that can attach to runtime, driver, and device events.

The conceptual analogy is:

$$\text{Linux kernel observability} \iff \text{GPU stack observability}$$

and:

$$\text{eBPF for kernel hooks} \iff \text{eBPF-style programs for GPU driver and device hooks}$$

Hardware-Level Explanation

SMs, Warps, Thread Blocks, and Scheduling

A CUDA kernel is launched as a grid of thread blocks. Each block is assigned to an SM. Threads inside a block are grouped into warps, usually of size 32.

Let a grid have B blocks and each block have T threads. The number of warps per block is:

$$W_{\text{block}} = \left\lceil \frac{T}{32} \right\rceil$$

The total number of warps is:

$$W_{\text{total}} = B \cdot W_{\text{block}}$$

If the GPU has S SMs, then blocks are distributed over SMs subject to resource constraints such as registers, shared memory, maximum resident blocks, and maximum resident warps.

A simplified resource-bound occupancy model is:

$$B_{\text{resident}} = \min\left(B_{\max}, \left\lfloor \frac{R_{\text{SM}}}{R_{\text{block}}} \right\rfloor, \left\lfloor \frac{M_{\text{SM}}}{M_{\text{block}}} \right\rfloor, \left\lfloor \frac{W_{\text{SM}}}{W_{\text{block}}} \right\rfloor\right)$$

where:

- R_{SM} is registers available per SM,
- R_{block} is registers required per block,
- M_{SM} is shared memory available per SM,
- M_{block} is shared memory required per block,
- W_{SM} is the warp residency limit,
- $B_{\max}$ is the architectural block residency limit.

This matters because a kernel can be submitted and even visible in a high-level trace while some of its blocks are still waiting for SM resources.

Why CUPTI Kernel Time May Not Explain the Real Delay

Suppose a CPU launches a kernel at time t_{cpu}. CUPTI-like tracing may report kernel start and finish times:

$$[t_{\text{start}}, t_{\text{end}}]$$

However, inside the kernel, not all blocks necessarily begin at t_{start}. Let block b begin executing on an SM at time t_b. Then the hidden block scheduling spread is:

$$\Delta_{\text{block-spread}} = \max_b t_b - \min_b t_b$$

A large $\Delta_{\text{block-spread}}$ indicates that many blocks waited for SM resources, perhaps due to concurrency, multi-stream execution, multi-process contention, or occupancy limits.

A device-side instrumentation system can report:

$$\{(b, \text{SM}(b), t_b)\}_{b=1}^{B}$$

which is richer than only knowing:

$$(t_{\text{start}}, t_{\text{end}})$$

SIMT Execution and eBPF Overhead

A CPU eBPF program normally executes in a scalar CPU context. GPUs execute in SIMT fashion, where a warp of threads executes a common instruction stream. If an injected eBPF program branches differently per thread, it can cause warp divergence.

Let a warp contain lanes $0, \ldots, 31$. If branch predicate p_i differs across lanes, the warp may serialize both paths:

$$T_{\text{diverged}} \approx T_{\text{path A}} + T_{\text{path B}}$$

instead of:

$$T_{\text{uniform}} \approx \max\left(T_{\text{path A}}, T_{\text{path B}}\right)$$

Therefore, a GPU eBPF design must be careful to avoid excessive per-thread instrumentation.

The lecture highlights optimizations such as:

- execute instrumentation once per warp rather than once per thread;
- execute instrumentation once per thread block for block-level events;
- use warp-uniform control flow where possible;
- place maps in GPU memory when device-side accesses dominate;
- duplicate maps when both CPU and GPU need efficient access.

Memory Placement for eBPF Maps

eBPF maps are essential because they store observations and share state. On GPUs, map placement strongly affects overhead.

Let L_{GPU} be access latency to GPU-local memory and L_{CPU} be access latency from GPU to CPU-host memory. Usually:

$$L_{\text{CPU}} \gg L_{\text{GPU}}$$

If an injected device-side program updates a map on every warp or memory access, placing that map in host memory can be catastrophic.

A placement policy can be modeled as:

$$\text{place}(M) = \begin{cases} \text{GPU memory}, & \text{if device writes dominate,} \\ \text{CPU memory}, & \text{if host reads dominate and update rate is low,} \\ \text{replicated}, & \text{if both CPU and GPU need frequent access.} \end{cases}$$

Visual Concept

Draw two map placements. In the first, GPU SMs update a map in CPU memory across PCIe, causing high latency. In the second, SMs update a GPU-local map, and the CPU periodically drains or snapshots it.

Mathematical Reasoning

Observability as a Cross-Layer Event Join

A major theme is correlating CPU and GPU information. Let host-side events be:

$$H = \left\{ (h_i, t_i^H, c_i) \right\}_{i=1}^{N}$$

where h_i is a host event type, t_i^H is a host timestamp, and c_i is context such as process ID, thread ID, or call stack.

Let device-side events be:

$$D = \left\{ (d_j, t_j^D, k_j, s_j) \right\}_{j=1}^{M}$$

where d_j is a device event type, t_j^D is a device timestamp, k_j is a kernel identifier, and s_j is SM or warp metadata.

The observability problem is to construct a joined trace:

$$J = H \bowtie D$$

using shared keys such as process, CUDA stream, kernel launch ID, GPU channel, timestamp correlation, or injected metadata.

The resulting joined event can answer:

Which CPU call chain caused this device-side event?

and:

Which GPU-side scheduling delay corresponds to this host-side launch?

Launch Latency Decomposition

A kernel launch delay can be decomposed as:

$$T_{\text{observed}} = T_{\text{CPU-runtime}} + T_{\text{driver-submit}} + T_{\text{queue-wait}} + T_{\text{SM-wait}} + T_{\text{device-exec}}$$

where:

- $T_{\text{CPU-runtime}}$ is overhead in the CPU runtime path,
- $T_{\text{driver-submit}}$ is driver submission overhead,
- $T_{\text{queue-wait}}$ is time waiting in GPU command queues,
- $T_{\text{SM-wait}}$ is time waiting for SM resources,
- $T_{\text{device-exec}}$ is actual device execution time.

A profiler with only kernel start and end time mostly sees:

$$T_{\text{kernel}} = t_{\text{end}} - t_{\text{start}}$$

But an eBPF-style cross-layer trace aims to estimate each term separately.

Memory-Bound Versus Compute-Bound Reasoning

The lecture's workload examples motivate memory behavior analysis. A standard roofline quantity is arithmetic intensity:

$$I = \frac{\text{FLOPs}}{\text{Bytes moved}}$$

The achievable performance is bounded by:

$$P \leq \min\left(P_{\text{peak}}, B_{\text{mem}} \cdot I\right)$$

where:

- P_{peak} is peak compute throughput,
- B_{mem} is memory bandwidth,
- I is arithmetic intensity.

If $B_{\text{mem}} \cdot I < P_{\text{peak}}$, the workload is memory-bound. If $P_{\text{peak}} < B_{\text{mem}} \cdot I$, the workload is compute-bound.

Observability helps determine whether poor performance comes from low arithmetic intensity, poor coalescing, UVM faults, scheduling delays, or CPU-side launch overhead.

UVM Policy Model

A UVM policy can be modeled as a decision function:

$$\pi_{\text{UVM}} : (P, \mathcal{H}(P), M_{\text{free}}, Q_{\text{fault}}, \mathcal{W}) \rightarrow \{\text{migrate}, \text{evict}, \text{prefetch}, \text{keep}\}$$

where:

- P is the page or block being considered,
- $\mathcal{H}(P)$ is its access history,
- M_{free} is available GPU memory,
- Q_{fault} is the current page fault queue,
- $\mathcal{W}$ represents workload context.

Hard-coded policies limit the expressiveness of π_{UVM}. Driver-side eBPF attempts to make parts of this policy programmable while preserving safety.

Scheduling Policy Model

A GPU driver scheduling policy can be represented as:

$$\pi_{\text{sched}} : (G, p, q, \tau, \ell) \rightarrow (\tau', \ell', a)$$

where:

- G is a GPU scheduling object such as a channel or TSG,
- p is process or tenant identity,

- q is queue state,
- τ is the current time slice,
- ℓ is interleave or priority level,
- τ' and ℓ' are updated scheduling parameters,
- a is an action such as admit, delay, reject, or reprioritize.

Driver-side programmable hooks can expose a narrow safe interface for adjusting τ, ℓ, or admission behavior without allowing arbitrary unsafe driver modification.

Code Walkthrough

Minimal CPU eBPF-Style Program

The lecture describes eBPF as small C programs attached to kernel or user-space hook points. A simplified example is shown below.

```
struct event_t {
    unsigned long pid;
    unsigned long timestamp_ns;
};

BPF_HASH(counts, unsigned long, unsigned long);
BPF_PERF_OUTPUT(events);

int on_cuda_launch(struct pt_regs *ctx) {
    unsigned long pid = bpf_get_current_pid_tgid();
    unsigned long ts = bpf_ktime_get_ns();
    unsigned long zero = 0;
    unsigned long *cnt;

    cnt = counts.lookup_or_init(&pid, &zero);
    if (cnt) {
        (*cnt) += 1;
    }

    struct event_t event = {};
    event.pid = pid;
    event.timestamp_ns = ts;
    events.perf_submit(ctx, &event, sizeof(event));
    return 0;
}
```

This code illustrates several core eBPF ideas:

- `BPF_HASH` creates a key-value map.

- `BPF_PERF_OUTPUT` creates an event output channel to user space.
- `bpf_get_current_pid_tgid` obtains process identity.
- `bpf_ktime_get_ns` obtains a timestamp.
- The program records how many launch-like events were observed per process.

In a GPU observability setting, this host-side event can be joined with device-side events to compute launch-to-SM latency.

Representative Device-Side Probe Logic

The lecture discusses compiling eBPF-like probe logic into PTX and injecting it into GPU kernels. The following CUDA-like pseudocode shows what a block-entry probe might conceptually do.

```
extern "C" __device__
void probe_block_entry(unsigned long long *sm_counts,
                       unsigned long long *block_times) {
    unsigned int smid;
    asm volatile("mov.u32 %0, %smid;" : "=r"(smid));

    unsigned long long t = clock64();
    unsigned int bid = blockIdx.x;

    if (threadIdx.x == 0) {
        atomicAdd(&sm_counts[smid], 1ULL);
        block_times[bid] = t;
    }
}
```

The important hardware behavior is:

- The probe reads the SM identifier using inline assembly.
- It records a device timestamp using `clock64`.
- Only one thread per block performs the update, avoiding a per-thread overhead explosion.
- `atomicAdd` accumulates how many blocks were scheduled on each SM.
- The resulting array can be used to build an SM load-balance histogram.

If every thread executed the atomic update, the overhead would scale as:

$$O(B \cdot T)$$

where B is the number of blocks and T is threads per block. Restricting the update to one thread per block reduces this to:

$$O(B)$$

This is crucial for making online instrumentation practical.

Warp-Level Probe Optimization

A warp-level probe can reduce overhead further for events where one observation per warp is enough.

```
extern "C" __device__
void probe_warp_event(unsigned long long *warp_counter) {
    unsigned int lane = threadIdx.x & 31;
    unsigned int warp_id = threadIdx.x >> 5;

    if (lane == 0) {
        atomicAdd(&warp_counter[warp_id], 1ULL);
    }
}
```

This code uses the fact that for a warp size of 32:

$$\text{lane} = \texttt{threadIdx.x} \bmod 32$$

and:

$$\text{warp\_id} = \left\lfloor \frac{\texttt{threadIdx.x}}{32} \right\rfloor$$

Only lane 0 performs the update. This creates a single atomic operation per warp rather than thirty-two atomic operations.

PTX Injection Concept

The lecture explains that the system can obtain original PTX from a CUDA fat binary, compile the eBPF program into PTX, and inject calls into the original kernel's PTX.

A simplified PTX-level idea is:

```
.visible .entry original_kernel(...) {
    // Original prologue

    call.uni probe_block_entry;
```

```
    // Original kernel body

    call.uni probe_block_exit;

    ret;
}
```

The injected calls behave like inline hooks. The original kernel executes, jumps to probe code, records information, and then returns to the original program.

Conceptually:

Original PTX + Probe PTX → Instrumented PTX → SASS → GPU execution

The lecture notes that one pragmatic implementation path is string-based PTX rewriting: insert a call at selected PTX locations, then invoke the NVIDIA toolchain again to assemble the modified PTX into executable device code.

Runtime Instrumentation Pipeline

The lecture describes a runtime architecture similar to `bpftime`. A simplified flow is:

1. Write an eBPF-style program.
2. Compile the program into bytecode or an intermediate representation.
3. Load the program and metadata into shared memory.
4. Attach a runtime library into the target process through `LD_PRELOAD` or runtime injection.
5. Hook CUDA runtime loading paths.
6. Extract or decompile PTX from the fat binary.
7. Compile the eBPF probe into PTX.
8. Inject PTX calls into the target kernel.
9. Reassemble and load the instrumented GPU kernel.
10. Collect device events into maps or buffers.

The important point is that the application does not need to be manually rewritten. Instrumentation is attached dynamically.

Driver-Side eBPF

GPU Channels and TSGs

The lecture discusses GPU scheduling through driver-visible abstractions. A user-space CUDA runtime submits work through command buffers and queues. The driver uses scheduling objects such as GPU channels and TSGs.

A simplified path is:

CUDA kernel launch → push buffer → doorbell → GPU channel → TSG → firmware runlist → SM execution

A **GPU channel** can be thought of as a command queue associated with GPU work submission. A **TSG**, or thread scheduling group, is a scheduling unit managed by the driver and firmware. Firmware may schedule TSGs using runlists.

The driver may not directly schedule individual thread blocks. Instead, it controls higher-level scheduling objects and parameters such as time slice, priority, and interleave behavior.

Why Driver-Side Hooks Matter

User-space frameworks can optimize within their own process, but they often lack global visibility. The driver can see:

- multiple processes;
- multiple tenants;
- memory residency state;
- GPU channels and scheduling objects;
- UVM page fault events;
- device memory pressure;
- cross-application contention.

This makes the driver a powerful place for observability and policy. However, arbitrary driver modification is dangerous. The lecture therefore emphasizes narrow, safe programmable interfaces.

UVM Block Interface

A driver-side eBPF interface for UVM can expose page or block operations in a controlled way. For example, it may allow a policy to move a block to the head or tail of an internal list.

A programmable cache-like model is:

$$\text{on\_access}(B) \rightarrow \text{update position of } B$$

$$\text{on\_evict}() \rightarrow \arg\min_{B \in \mathcal{C}} \text{priority}(B)$$

where B is a GPU memory block and $\mathcal{C}$ is the current GPU-resident cache of blocks.

A simple least-recently-used score is:

$$\text{priority}_{\text{LRU}}(B) = t_{\text{last access}}(B)$$

An alternative frequency-aware score is:

$$\text{priority}_{\text{LFU}}(B) = \text{access count}(B)$$

A custom eBPF policy could combine recency and frequency:

$$\text{score}(B) = \alpha \cdot \frac{1}{1 + t_{\text{now}} - t_{\text{last access}}(B)} + \beta \cdot \log\left(1 + f(B)\right)$$

where $f(B)$ is access frequency and α, β tune the policy.

Prefetching for LLM and ML Workloads

The lecture notes that hardware and workload behavior evolve faster than fixed software policies. LLM inference and ML training can have structured memory movement patterns that generic UVM policies may not exploit.

For a sequential access pattern, a prefetch policy may choose:

$$\text{prefetch}\left(P_{i+1}, P_{i+2}, \ldots, P_{i+k}\right)$$

after observing an access to page P_i. For KV-cache access, prefetching may depend on token position, batch, layer, and head layout.

A programmable prefetch policy can be modeled as:

$$\pi_{\text{prefetch}}\left(P_i, \mathcal{H}, \mathcal{K}\right) = \{P_j : j \in \mathcal{N}(i, \mathcal{H}, \mathcal{K})\}$$

where:

- $\mathcal{H}$ is observed access history,

- $\mathcal{K}$ is workload-specific knowledge,
- $\mathcal{N}$ predicts the next useful pages.

Performance Intuition

Why Online Device Observability Is Useful

Traditional profilers are often offline or coarse-grained. Device-side eBPF-style instrumentation aims to provide online and programmable observations. This is valuable when:

- the workload is long-running;
- behavior changes over time;
- multiple tenants share a GPU;
- kernel launches are frequent and small;
- CPU and GPU events must be correlated;
- policy adaptation should happen during execution.

Main Sources of Overhead

The lecture highlights several overhead sources.

Instrumentation Frequency

If a probe runs per instruction or per thread, overhead can be enormous. If it runs per warp or per block, overhead may be acceptable.

$$O(\text{instructions} \cdot \text{threads}) \gg O(\text{warps}) \gg O(\text{blocks})$$

Warp Divergence

Branchy probe logic can cause SIMT serialization. A good GPU probe should use warp-uniform behavior whenever possible.

Atomic Contention

If many threads update the same counter, atomic contention can dominate.

For N updates to one global counter, the cost is approximately:

$$T_{\text{atomic}} \approx N \cdot L_{\text{atomic-contention}}$$

Reducing updates from per-thread to per-warp changes N by approximately $32\times$:

$$N_{\text{warp}} \approx \frac{N_{\text{thread}}}{32}$$

Map Placement

GPU-to-host map updates are expensive. GPU-local maps reduce device overhead but require explicit host-side draining or synchronization.

Why Driver-Side Policies Can Improve UVM

UVM policies determine when to migrate, evict, and prefetch memory. A poor policy can cause repeated page faults:

$$T_{\text{total faults}} = \sum_{i=1}^{F} T_{\text{fault},i}$$

If a custom policy reduces the number of faults from F to F', then the saved time is:

$$\Delta T = \sum_{i=1}^{F} T_{\text{fault},i} - \sum_{i=1}^{F'} T_{\text{fault},i}$$

When $F' \ll F$, the improvement can be large, especially for memory-offloaded workloads.

Common Misconceptions

Misconception: eBPF on GPUs Means Replacing CUDA

eBPF-style GPU instrumentation is not a replacement for CUDA programming. It is a mechanism for injecting observation or policy logic around existing execution paths.

CUDA still defines the main computation:

$$\text{input tensors} \rightarrow \text{CUDA kernel} \rightarrow \text{output tensors}$$

The eBPF-style probe observes or adjusts metadata:

$$\text{kernel event} \rightarrow \text{probe} \rightarrow \text{map update or policy decision}$$

Misconception: Observability Automatically Improves Performance

Observability provides information. Performance improvement requires using that information to change code, configuration, scheduling, memory placement, or policy.

$$\text{measurement} \not\Rightarrow \text{speedup}$$

The chain is:

$$\text{measurement} \rightarrow \text{diagnosis} \rightarrow \text{policy or implementation change} \rightarrow \text{speedup}$$

Misconception: CUPTI, NVBit, and eBPF Solve the Same Problem

They overlap but are not identical.

- CUPTI exposes standard profiling and tracing interfaces.
- NVBit-like systems support dynamic binary instrumentation.
- eBPF-style systems emphasize verified, programmable, dynamically attached logic with map-based communication and policy hooks.

Misconception: Per-Thread Probes Are Usually Fine

On GPUs, per-thread instrumentation can multiply overhead by thousands or millions. Probe granularity must match the question being asked.

If the question is SM load balance, one event per block may be enough. If the question is memory instruction behavior, sampling may be necessary.

Misconception: User-Space Frameworks Can Fully Solve Multi-Tenant GPU Policy

A single framework sees its own workload. The driver sees multiple processes and tenants. Therefore, some scheduling and memory policies are naturally driver-level concerns.

Practical Takeaways

1. GPU observability must be cross-layer. CPU launch traces, driver state, UVM events, and device-side execution all matter.

2. Kernel start and end times are not enough to explain many latency problems.
3. Fine-grained device probes can reveal SM assignment, block scheduling delays, warp behavior, and memory access behavior.
4. eBPF's most important ideas are dynamic attachment, safety verification, maps, helpers, and programmable hooks.
5. GPU eBPF is harder than CPU eBPF because SIMT execution introduces warp divergence, coalescing concerns, atomic contention, and memory placement issues.
6. Probe granularity is a first-order performance design decision.
7. Driver-side programmable hooks can expose safe customization points for UVM and scheduling policies.
8. A narrow safe interface is preferable to arbitrary driver modification.
9. UVM offloading workloads can benefit from workload-aware eviction and prefetch policies.
10. Multi-tenant GPU systems need observability and scheduling control at a level deeper than a single user-space framework.

Project or Experiment Ideas

Experiment 1: Block-to-SM Histogram

Implement a CUDA kernel that records which SM each block runs on. Compare SM distribution for different grid sizes, block sizes, register usage, and shared memory usage.

Measure:

$$\text{imbalance} = \frac{\max_s C_s - \min_s C_s}{\frac{1}{S}\sum_{s=1}^{S} C_s}$$

where C_s is the number of blocks scheduled on SM s.

Experiment 2: Per-Thread Versus Per-Warp Probe Overhead

Write two versions of an instrumentation counter:

- every thread performs an atomic increment;
- only lane 0 of each warp performs an atomic increment.

Compare runtime overhead as a function of grid size.

Expected result:

$$T_{\text{per-thread}} \gg T_{\text{per-warp}}$$

especially when atomics target the same memory location.

Experiment 3: UVM Page Fault Microbenchmark

Allocate managed memory larger than GPU memory or force CPU residency before GPU access. Measure first-touch latency, repeated access latency, and prefetching effects.

Compare:

- no prefetch;
- explicit `cudaMemPrefetchAsync`;
- sequential access;
- random access;
- strided access.

Experiment 4: Launch Latency Decomposition

Use CPU-side timestamps around CUDA launches and device-side timestamps inside kernels. Estimate:

$$t_{\text{device-entry}} - t_{\text{host-launch}}$$

Then run multiple streams or multiple processes to observe whether launch-to-entry delay grows due to contention.

Experiment 5: Cross-Layer Trace Viewer

Build a simple trace format containing:

- CPU function events;
- CUDA launch events;
- kernel identifiers;
- block-entry timestamps;
- SM identifiers;
- UVM page fault events.

Visualize the joined trace as a timeline.

Final Mental Model

GPU observability is not just about measuring kernel duration. Modern GPU workloads pass through a deep stack: Python or C++ framework code, CUDA runtime, GPU driver, firmware, GPU queues, SM scheduling, memory hierarchy, and UVM migration. A delay or bottleneck can occur at any layer.

The lecture's core idea is to bring the eBPF philosophy to GPUs:

$$\text{safe} + \text{dynamic} + \text{programmable} + \text{cross-layer}$$

For device execution, this means injecting verified probe logic into GPU kernels through PTX-level instrumentation and collecting fine-grained events such as SM assignment, block scheduling, memory access, and device timestamps. For driver behavior, it means adding narrow safe hooks that allow policies for UVM migration, prefetching, eviction, and GPU scheduling to be observed and customized.

The final engineering mental model is:

> A GPU is not merely an accelerator running kernels. It is a distributed execution system with queues, drivers, firmware, memory migration, scheduling objects, SM resource constraints, and workload-specific access patterns. eBPF-style GPU observability turns that hidden system into something measurable, explainable, and eventually programmable.